

Data Science Tools And Software

Dr. Laila M. Fatehy

Department Of Mathematics And Computer Science
Alexandria University

Use case: Data Science in Health Care

Diabetes Dataset

- This dataset is originally from the National Institute of Diabetes and Digestive and Kidney Diseases. The objective is to predict based on diagnostic measurements whether a patient has diabetes.
- Diabetes dataset is one of the datasets available in sklearn.
- It consists of 10 baseline variables, age, sex, body mass index, average blood pressure, and six blood serum measurements were obtained for each of $n = 442$ diabetes patients, as well as the response of interest, a quantitative measure of disease progression one year after baseline.
- The population lives near Phoenix, Arizona, USA
- The task is to come up with Linear Regression training algorithm and thus predictions for the test dataset given.

Data Set Characteristics:

Number of Instances: 442

Number of Attributes: First 10 columns are numeric predictive values

Target: Column 11 is a quantitative measure of disease progression one year after baseline

Attribute Information:

- age: age in years
- sex
- bmi: body mass index
- bp: average blood pressure
- s1: tc, T-Cells (a type of white blood cells)
- s2: ldl, low-density lipoproteins
- s3: hdl, high-density lipoproteins
- s4: tch, thyroid stimulating hormone
- s5: ltg, lamotrigine
- s6: glu, blood sugar level

Regression Example - Linear Regression

```
from sklearn import datasets
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import mean_squared_error, r2_score
import pandas as pd

# Load the diabetes dataset from sklearn
diabetes = datasets.load_diabetes()
diabetes
# Display basic information about the dataset
print(diabetes.DESCR)

# columns
diabetes.feature_names

# Features and target variable
X=diabetes.data
Y=diabetes.target
X.shape, Y.shape
print(f"X.shape : {X.shape}")
print(f"Y.shape: {Y.shape}")

# Split the data into training and testing sets
X_train, X_test, Y_train, Y_test = train_test_split(X,Y,test_size=0.3,random_state=99)
X_train.shape, Y_train.shape
print(f"X_train.shape : {X_train.shape}")
print(f"Y_train.shape: {Y_train.shape}")

#Linear Regression
linreg = LinearRegression()
linreg.fit(X_train,Y_train)
print(linreg.coef_)

#Predict the response for test dataset
Y_pred=linreg.predict(X_test)
result = pd.DataFrame({'Actual': Y_test, 'Predict' : Y_pred})
result

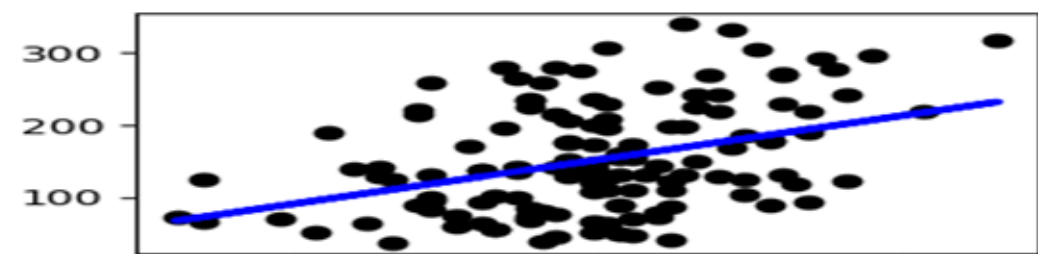
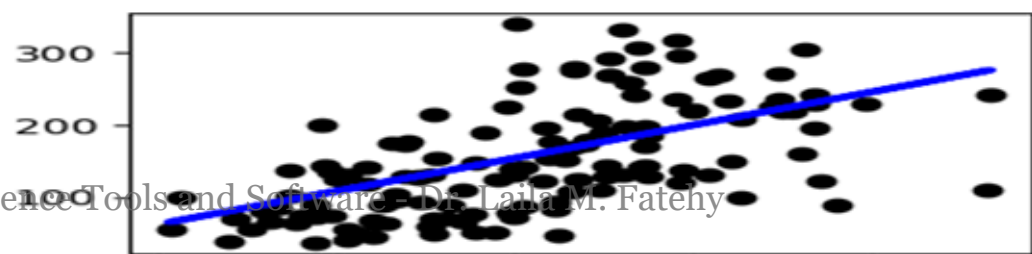
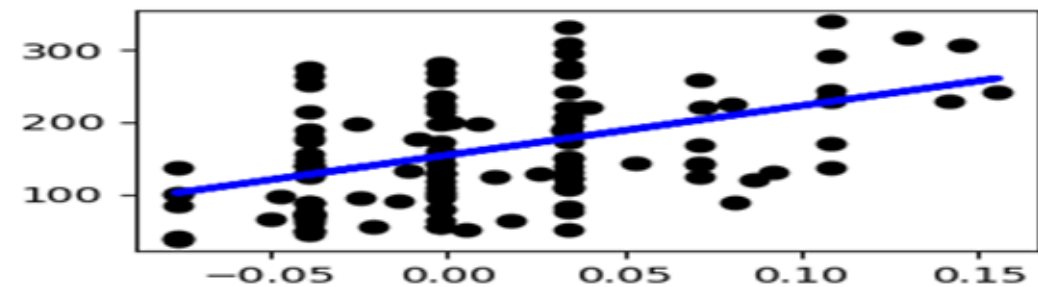
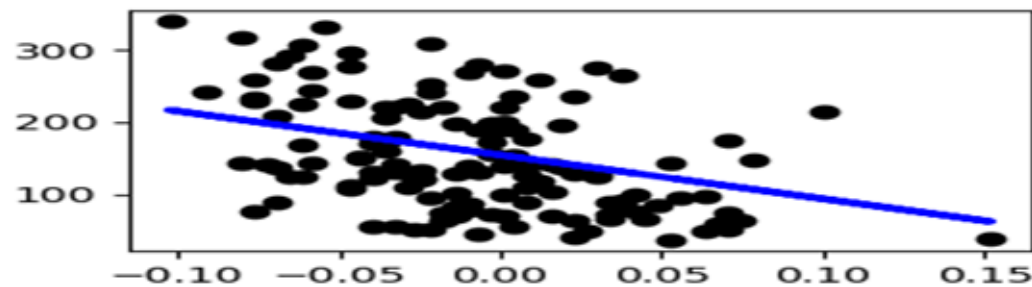
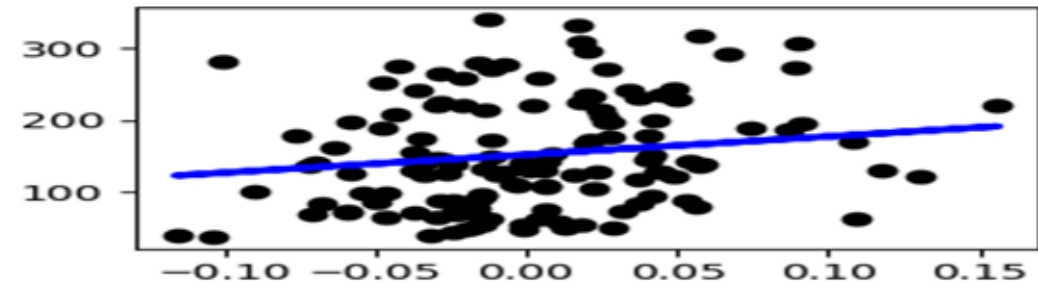
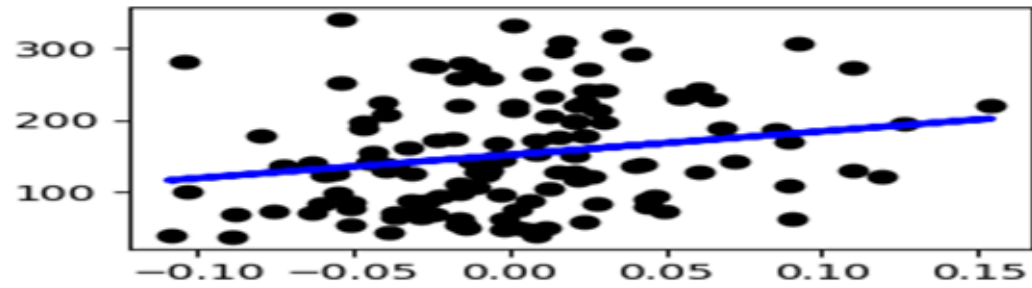
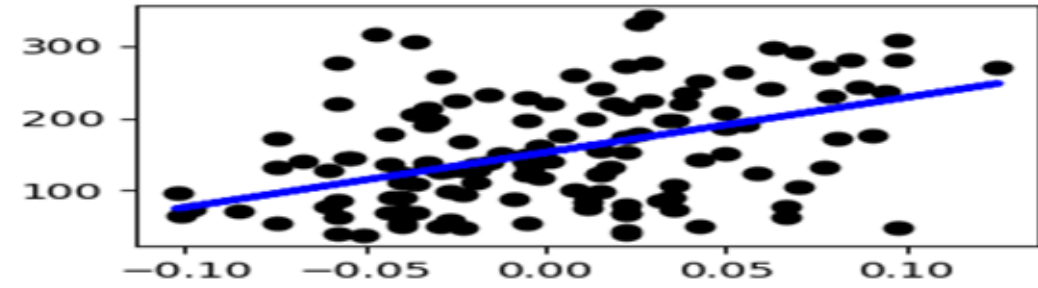
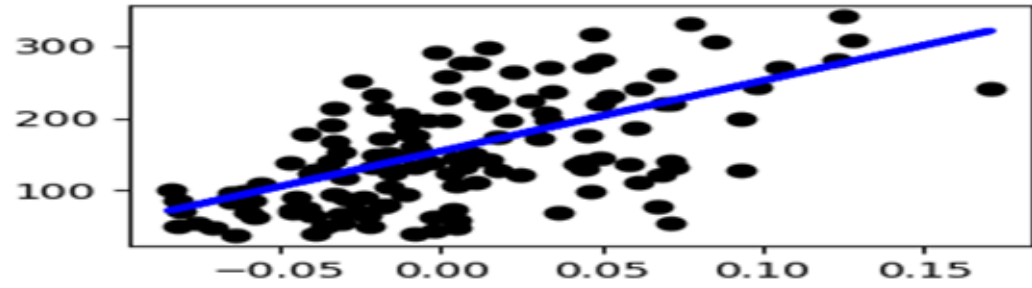
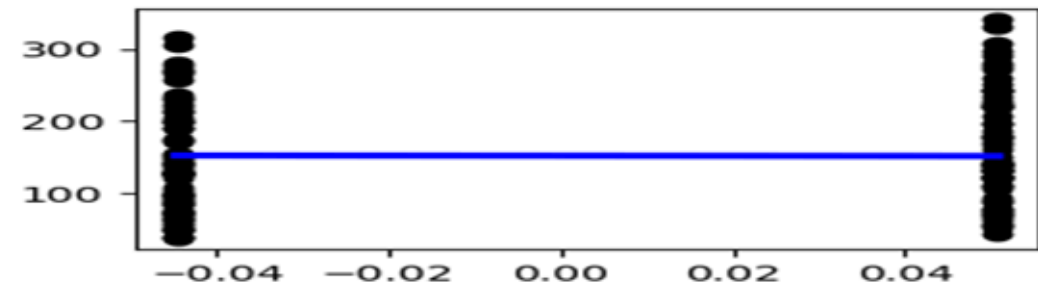
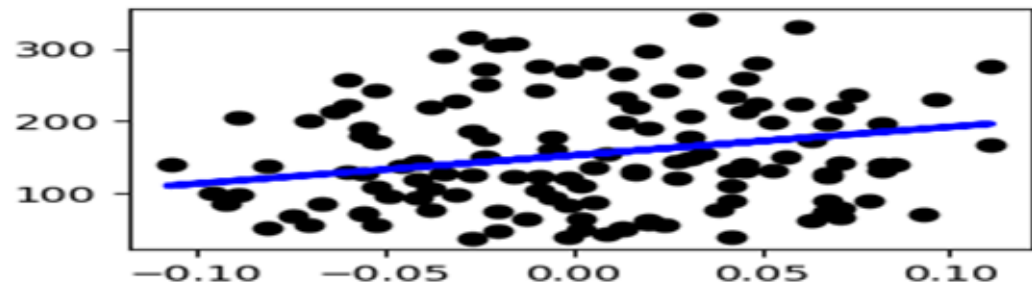
# Model Accuracy, how often is the classifier correct?
# Evaluating the model
mse = mean_squared_error(Y_test,Y_pred)
r2 = r2_score(Y_test,Y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}") # or linreg.score(X_test, Y_test) # R-squared
```

Regression Example - Linear Regression

Actually, you have ten physiological factors in the diabetes dataset. Therefore, to have a more complete picture of all the training set, you can create a linear regression for every physiological feature, creating ten models and seeing the result for each of them through a linear chart.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import linear_model
from sklearn import datasets
diabetes = datasets.load_diabetes()
X_train, X_test, Y_train, Y_test = train_test_split(X,Y,test_size=0.3,random_state=99)
plt.figure(figsize=(8,12))
for f in range(0,10):
    xi_test = X_test[:,f]
    xi_train = X_train[:,f]
    xi_test = xi_test[:,np.newaxis]
    xi_train = xi_train[:,np.newaxis]
    linreg.fit(xi_train,Y_train)
    y = linreg.predict(xi_test)
    plt.subplot(5,2,f+1)
    plt.scatter(xi_test,Y_test,color='k')
    plt.plot(xi_test,y,color='b',linewidth=3)
```

Data Science Tools

WEKA

Functionality

- Weka is tried and tested open source machine learning software that can be accessed through a graphical user interface, standard terminal applications, or a Java API.
- Developed by the University of Waikato, it is widely used for teaching, research, and industrial applications, and contains built-in tools for standard machine learning tasks.
- You can use Weka to build machine learning models, train classifiers, and run evaluations without having to write a single line of code. In addition, Weka provides transparent access to well-known data science toolboxes such as scikit-learn and R

Platform

- Windows
- MacOS
- Linux



Weka13-2-data-analysis

https://www.tutorialspoint.com/weka/weka_tutorial.pdf

Data processing and Machine Learning Algorithms in Weka

Explorer: pre-processing the data

- Data can be imported from a file in various formats: ARFF, CSV, C4.5, binary
- Data can also be read from a URL or from an SQL database (using JDBC)
- Pre-processing tools in WEKA are called “filters”
- WEKA contains filters for:
Discretization, normalization, resampling, attribute selection, transforming and combining attributes, ...

Weka Explorer

Preprocess | Classify | Cluster | Associate | Select attributes | Visualize

Open file... | Open URL... | Open DB... | Generate... | Undo | Edit... | Save...

Filter

- filters
 - AllFilter
 - MultiFilter
 - supervised
 - attribute
 - Add
 - AddCluster
 - AddExpression
 - AddID
 - AddNoise
 - ChangeDateFormat
 - ClusterMembership
 - Copy
 - Discretize
 - FirstOrder
 - MakeIndicator
 - MathExpression
 - MergeTwoValues
 - MultiInstanceToPropositional
 - NominalToBinary
 - Normalize
 - NumericToBinary
 - NumericTransform
 - Obfuscate

Selected attribute

Name: duration
Missing: 1 (2%)
Distinct: 3
Type: Numeric
Unique: 0 (0%)

Statistic	Value
Minimum	1
Maximum	3
Mean	2.161
StdDev	0.708

Class: class (Nom) | Visualize All

17 | class

10 27 19

Data Filtering

WEKA File Formats and Data Preprocessing

Weka-file formats-preprocessing

Explorer: building “classifiers”

- **Classifiers in WEKA are models for predicting nominal or numeric quantities**
- **Implemented learning schemes include:**
 - **Decision trees and lists, instance-based classifiers, support vector machines, multi-layer perceptrons, logistic regression, Bayes’ nets, ...**

Test options

☐ Use training set☐ Supplied test set

Set...

☒ Cross-validation

Folds

10

☐ Percentage split

%

66

More options...

Classifier output

Classification

(Nom) play



Start

Stop

Result list (right-click for options)

Linear Machine Learning Algorithms

Linear algorithms assume that the predicted attribute is a linear combination of the input attributes.

- Linear Regression: `function.LinearRegression`
- Logistic Regression: `function.Logistic`

Nonlinear Machine Learning Algorithms

Nonlinear algorithms do not make strong assumptions about the relationship between the input attributes and the output attribute being predicted.

- Naive Bayes: `bayes.NaiveBayes`
- Decision Tree (specifically the C4.5 variety): `trees.J48`
- k-Nearest Neighbors (also called KNN: `lazy.IBk`)
- Support Vector Machines (also called SVM): `functions.SMO`
- Neural Network: `functions.MultilayerPerceptron`

Linear and Quadratic discriminant analysis

Install the `discriminantAnalysis` package

Package Manager

Official

Refresh repository cache Install Uninstall Toggle load

☐ Installed ☐ Available ☒ All ☐ Ignore dependencies/conflicts

Install/Uninstall/Refresh progress

Package	Category	Installed version
consistencySubsetEval	Attribute selection	
costSensitiveAttributeSelection	Attribute selection	
lagging	Ensemble learning	
decorate	Ensemble learning	
denormalize	Preprocessing	
discriminantAnalysis	Classification	1.0.4
distributedWekaBase	Distributed	
distributedWekaHadoop	Distributed	

Package search Clear

discriminantAnalysis: Classes for linear and quadratic discriminant analysis

URL: <http://weka.sourceforge.net/doc.packages/discriminantAnalysis>

Author: Eibe Frank

Maintainer: Web site: <http://www.cs.waikato.ac.nz/~frank/>

load iris.arff

Weka Explorer

Preprocess **Classify** Cluster Associate Select attributes Visualize

Classifier: Choose **LDA -R 1.0E-6**

Test options

☒ Use training set

☐ Supplied test set Set...

☐ Cross-validation Folds 10

☐ Percentage split % 66

More options...

(Nom) class

Start Stop

Result list (right-click for options)

03:17:12 - functions.LDA

Classifier output

Time taken to build model: 0 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0 seconds

=== Summary ===

Correctly Classified Instances	147	98	%
Incorrectly Classified Instances	3	2	%
Kappa statistic	0.97		
Mean absolute error	0.0179		
Root mean squared error	0.0971		
Relative absolute error	4.0336	%	
Root relative squared error	20.5977	%	
Total Number of Instances	150		

=== Detailed Accuracy By Class ===

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area
--	---------	---------	-----------	--------	-----------	-----	----------

Weka Explorer

Preprocess **Classify** Cluster Associate Select attributes Visualize

Classifier

Choose **QDA -R 1.0E-6**

Test options

☒ Use training set

☐ Supplied test set

☐ Cross-validation Folds

☐ Percentage split %

Classifier output

Time taken to build model: 0.01 seconds

=== Evaluation on training set ===

Time taken to test model on training data: 0.01 seconds

=== Summary ===

Correctly Classified Instances	147	98	%
Incorrectly Classified Instances	3	2	%
Kappa statistic	0.97		
Mean absolute error	0.0156		
Root mean squared error	0.0879		
Relative absolute error	3.5138	%	
Root relative squared error	18.6548	%	
Total Number of Instances	150		

=== Detailed Accuracy By Class ===

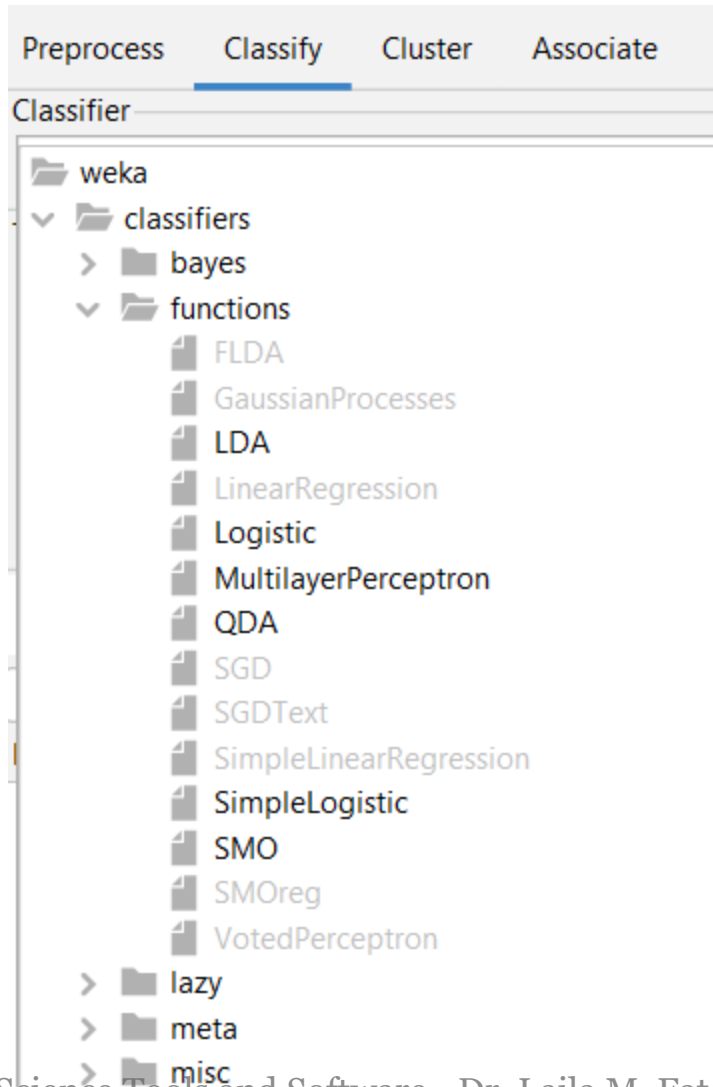
	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC
	1.000	0.000	1.000	1.000	1.000	1.000
	0.960	0.010	0.980	0.960	0.970	0.955
	0.980	0.020	0.961	0.980	0.970	0.955

(Nom) class

Result list (right-click for options)

- 03:17:12 - functions.LDA
- 03:19:03 - functions.QDA**

Neural Network



Choose

MultilayerPerceptron -L 0.3 -M 0.2 -N 500 -V 0

Test options

☒ Use training set
 ☐ Supplied test set
☐ Cross-validation Folds
☐ Percentage split %

(Nom) class

Result list (right-click for options)

14:39:48 - functions.MultilayerPerceptron
 14:40:07 - functions.MultilayerPerceptron
 14:41:04 - functions.MultilayerPerceptron
 14:41:17 - functions.MultilayerPerceptron

Classifier output

=== Run inf

Scheme:
 Relation:
 Instances:
 Attributes:

Test mode:

Neural Network

```

graph LR
    subgraph Input
        S1[sepalength]
        S2[sepalwidth]
        S3[petallength]
        S4[petalwidth]
    end
    subgraph Hidden
        H1(( ))
        H2(( ))
        H3(( ))
    end
    subgraph Output
        O1[Iris-setosa]
        O2[Iris-versicolor]
        O3[Iris-virginica]
    end
    S1 --- H1
    S1 --- H2
    S1 --- H3
    S2 --- H1
    S2 --- H2
    S2 --- H3
    S3 --- H1
    S3 --- H2
    S3 --- H3
    S4 --- H1
    S4 --- H2
    S4 --- H3
    H1 --- O1
    H1 --- O2
    H1 --- O3
    H2 --- O1
    H2 --- O2
    H2 --- O3
    H3 --- O1
    H3 --- O2
    H3 --- O3
  
```

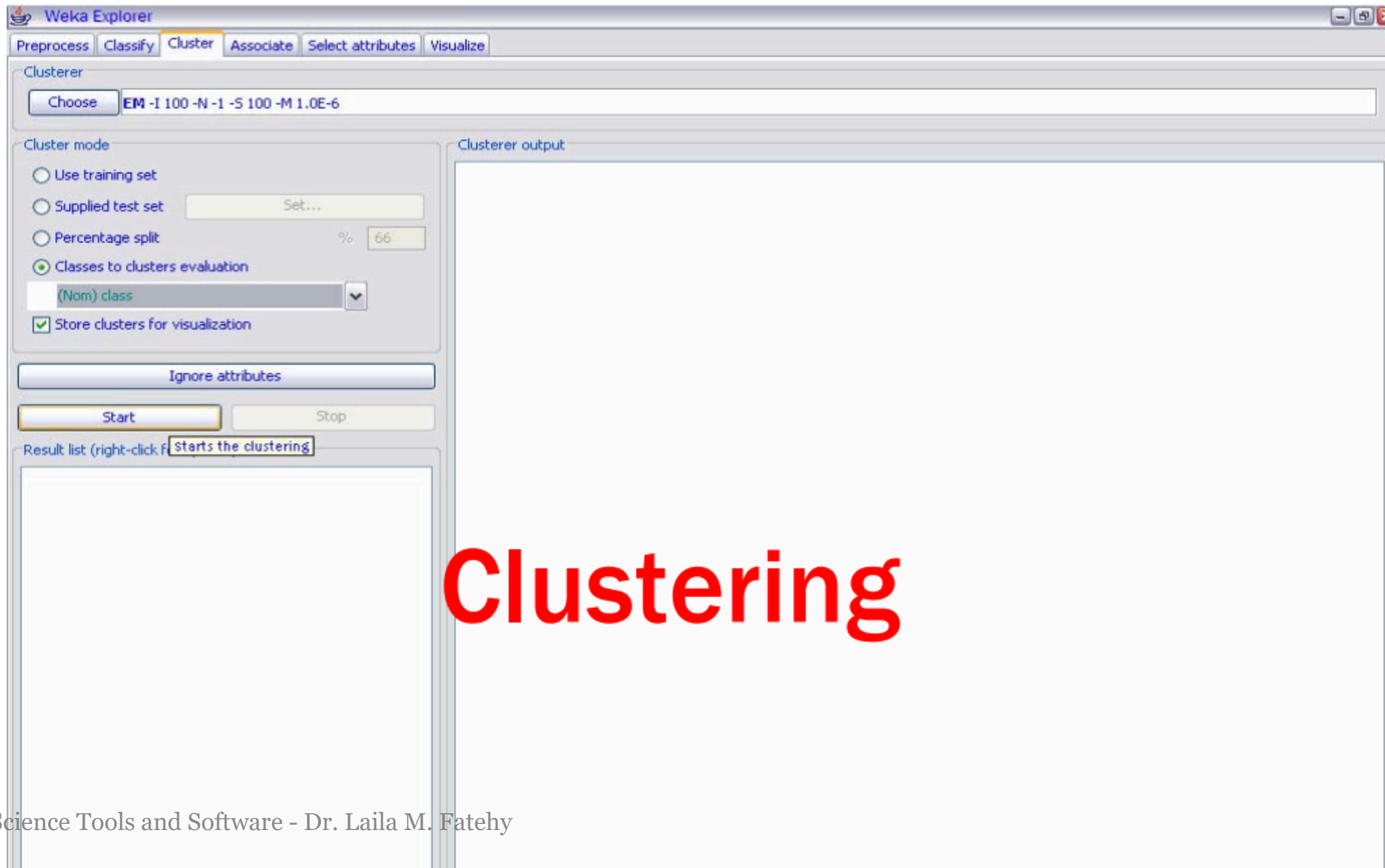
Controls

Epoch 0
 Num Of Epochs
 Error per Epoch = 0

Learning Rate =
 Momentum =

Explorer: clustering data

- WEKA contains “clusterers” for finding groups of similar instances in a dataset
- Implemented schemes are:
 - *k*-Means, EM, Cobweb, X-means, FarthestFirst
- Clusters can be visualized and compared to “true” clusters (if given)
- Evaluation based on loglikelihood if clustering scheme produces a probability distribution



Weka Explorer

Preprocess Classify **Cluster** Associate Select attributes Visualize Forecast

Clusterer

Choose **SimpleKMeans** -init 0 -max-candidates 100 -periodic-pruning 10000 -min-density 2.0 -t1 -1.25 -t2 -1.0 -N 2 -A "weka.core.EuclideanDistance" -R first

Cluster mode

☒ Use training set

☐ Supplied test set Set...

☐ Percentage split % 66

☐ Classes to clusters evaluation
(Nom) class v

☐ Store clusters for visualization

Ignore attributes

Start Stop

Result list (right-click for options)

14:36:37 - SimpleKMeans

Clusterer output

Within cluster sum of squared errors: 62.1436882815797

Initial starting points (random):

Cluster 0: 6.1,2.9,4.7,1.4,Iris-versicolor
Cluster 1: 6.2,2.9,4.3,1.3,Iris-versicolor

Missing values globally replaced with mean/mode

Final cluster centroids:

Attribute	Full Data (150.0)	Cluster# 0 (100.0)	1 (50.0)
sepalength	5.8433	6.262	5.006
sepalwidth	3.054	2.872	3.418
petallength	3.7587	4.906	1.464
petalwidth	1.1987	1.676	0.244
class	Iris-setosa	Iris-versicolor	Iris-setosa

